



**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
THAPATHALI CAMPUS**

**Minor Project Proposal
ON
GEO-LOCATING USERS IN MOUNTAINS BY DETECTING SKYLINES FROM
CAPTURED PHOTOS**

Submitted By:

Bibha Ray	(THA080BCT009)
Prasiddha Raj Poudel	(THA080BCT027)
Reema Kasula	(THA080BCT032)

Submitted To:

Department of Electronics and Computer Engineering
Thapathali Campus
Kathmandu, Nepal

June 2026

ABSTRACT

Navigation in Nepal's Khumbu region is often difficult when global positioning system signals fail in deep mountain valleys due to topographic shielding. This project proposes a visual positioning system that uses the mountain skyline to determine a user's location. The system will extract the silhouette from user's photograph and match them against a database of synthetic horizons. These reference views are generated using digital elevation models of the Himalayan landscape. A two-stage matching engine combining sliding-window Euclidean distance and Dynamic Time Warping (DTW) can resolve the coordinates even when the camera heading is unknown. This framework aims to provide a navigation alternative for travelers in remote environments.

Keywords—Visual Geolocation, Skyline Localization, Digital Elevation Models, Himalayan Topography, Sliding Window Search, Dynamic Time Warping, Khumbu Region.

TABLE OF CONTENTS

LIST OF FIGURES

1 INTRODUCTION

1.1 Background

Satellite navigation signals weaken or vanish in deep Himalayan valleys due to topographic shielding and multi-path interference [?, ?]. Canyon walls, steep ridges, and dense cloud cover block the line of sight to GNSS satellites, leaving trekkers and rescue teams without reliable coordinates. The mountain skyline, however, is always visible from any open viewpoint and its shape is tied directly to the observer’s position [?]. If a system can extract that shape from a photograph and match it against a known database of terrain profiles, it can determine where the photo was taken—without any satellite signal [?, ?]. Researchers proved this approach in European mountain ranges [?, ?] and hilly terrain in China [?].

1.2 Motivation

Navigation failure in the Khumbu region can be fatal. Trekkers routinely carry smartphones with cameras, and these cameras can serve as positioning sensors if the right software is available. An offline skyline-matching system would give users a backup that works even in GNSS-denied areas, independent of cell towers or internet connectivity.

1.3 Problem Statement

The goal is to estimate a camera’s geographic coordinates and viewing direction from a single mountain photograph. Three practical challenges make this difficult:

1. **Extraction:** Haze, clouds, and low contrast frequently obscure the skyline boundary [?].
2. **Search scale:** Without knowing which direction the camera is pointing, the system must search across all possible orientations for every candidate location in the database [?].
3. **Data scarcity:** Annotated mountain photographs are rare for the Himalayas, so synthetic training images must supplement real data [?].

1.4 Objectives

- Generate a 360-degree horizon database from the Copernicus GLO-30 Digital Elevation Model.
- Build a matching pipeline that combines Euclidean distance search with Dynamic Time Warping (DTW) to resolve coordinates from photographs.

1.5 Scope and Limitations

The study area covers a 40 km × 30 km region in the Khumbu valley of Nepal, bounded by:

- **Northwest (NW):** 86.582°E, 28.041°N
- **Northeast (NE):** 86.989°E, 28.041°N
- **Southwest (SW):** 86.582°E, 27.770°N
- **Southeast (SE):** 86.989°E, 27.770°N



Figure 1-1 Region of Interest (ROI)

The system is designed for offline use on standard portable hardware. It will not work when weather or obstacles completely block the skyline from view.

2 LITERATURE REVIEW

2.1 Evolution of Terrain Representation

For most of the twentieth century, topographic data came from field surveys and photogrammetric mapping [?]. These methods could not produce the dense elevation grids needed for automated skyline computation.

The Shuttle Radar Topography Mission (SRTM) in 2000 delivered the first near-global elevation dataset [?]. The ASTER Global DEM followed in 2009 [?]. Both datasets were useful but unreliable in steep terrain—radar shadows and layover artifacts introduced large errors around cliffs and deep valleys [?, ?], leading to inaccurate peak heights [?].

The Copernicus GLO-30 model, released by the European Space Agency (ESA) in 2020 [?, ?], resolved many of these problems. Independent evaluations confirm its superior vertical accuracy in steep mountain terrain [?]. At 30-meter resolution, GLO-30 captures the distinctive shapes of Himalayan peaks while remaining small enough to store and process on portable devices [?, ?].

2.2 Developments in Skyline Segmentation

Early skyline extraction methods in the 2000s relied on edge-detection filters that identified sharp changes in image intensity [?]. These filters performed poorly when haze, clouds, or shadows obscured the sky–terrain boundary [?].

The introduction of fully convolutional networks in 2015 marked a turning point for image segmentation [?]. Deep models could learn to distinguish sky from terrain even under poor lighting. However, the earliest networks were computationally heavy. Lighter architectures such as MobileUNet addressed this by using depthwise separable convolutions, which cut computation while preserving pixel-level accuracy [?]. More recent frameworks like YUNet adapt object detection to rapid sky segmentation and perform well under difficult weather conditions [?].

A recurring obstacle has been the scarcity of labeled training data. The GeoPose3K dataset, introduced by Brejcha and Cadik in 2017, provides over 3,000 alpine mountain images with precise camera pose labels [?]. Tomesek et al. later explored cross-modal rendering to generate additional synthetic training examples [?].

2.3 Progress in Silhouette Matching

Once a skyline is extracted, it must be matched against the reference database to determine the camera’s position and heading [?].

Early matching approaches compared profiles pixel by pixel, which was both slow and fragile—different camera lenses and viewpoints easily broke the alignment [?]. The method also assumed the camera’s heading was already known.

Rotational sliding-window search removed the heading requirement by testing the query against all possible angular offsets of each reference profile [?]. To make this tractable, researchers adopted two-stage pipelines. In 2013, Rakthanmanon et al. showed that Dynamic Time Warping (DTW) could be applied to fast subsequence matching [?]. DTW is tolerant of the stretching and compression caused by different focal lengths [?].

Full DTW search across a large database remains expensive [?]. Mueen et al. addressed this with a distance-profile algorithm that quickly eliminates poor candidates, leaving only a handful for DTW alignment [?]. This two-stage combination—fast filtering followed by elastic matching—makes real-time localization feasible on mobile hardware.

Baatz et al. introduced the first complete large-scale geolocation pipeline for mountains in 2012 [?]. The system used skylines from elevation models to find unknown camera headings across the whole Alps. Saurer et al. later released a journal extension that scaled this matching approach further [?]. The CrossLocate project later localized 39% of Alpine images within a one-kilometer radius [?]. The researchers tested all three systems exclusively in the European Alps.

2.4 Validation with Geotagged Imagery

Any alternative positioning system needs ground truth with reliable coordinates. Early studies used small, custom photo sets that lacked diversity in lighting and precise GPS tags.

The Google Street View Trekker campaign along the Everest trail [?] changed this. Captured in 2014, it contains over 45,000 high-resolution panoramas with accurate location and orientation metadata [?]. The Khumbu trails covered by this dataset provide a large, georeferenced collection of real mountain views suitable for systematic testing.

DEM-based skyline matching has been proven effective in mountainous terrain [?, ?, ?, ?, ?]. However, all existing systems target European or Chinese terrain. No system has been built or tested in the Himalayas, where extreme elevation, absent training data, and GPS failure in deep valleys present a distinct and unsolved problem. Table ?? summarizes existing approaches.

2.5 Research Gap

Existing systems work well in the Alps or in Chinese hills [?, ?, ?, ?, ?]. No researchers have built or tested these pipelines in Himalayan terrain. The Himalayas present unique

Table 2-1 Summary of existing skyline geolocation approaches

Authors	Year	Terrain	Matching Method
Baboud et al. [?]	2011	Alps	Edge cross-correlation
Baatz et al. [?]	2012	Alps (Switzerland)	Bag-of-curves
Saurer et al. [?]	2016	Alps	Contours + DTW
Pan et al. [?]	2020	China (hilly)	Euclidean + heatmap
Tomešek et al. [?]	2022	Alps	Synthetic rendering
This work	2026	Khumbu, Nepal	FFT + DTW

challenges including extreme elevation changes and an absence of labeled training data. This project adapts the established pipeline to Khumbu terrain using synthetic training data generated from the DEM.

3 METHODOLOGY

3.1 System Overview

Mountain peaks are permanent geographic features whose shapes, as seen from a given coordinate, are unique. A photograph taken in the Khumbu region encodes the observer’s position in the geometry of the visible skyline. This project reconstructs that geometry and matches it against a pre-computed reference database to recover the camera’s location and heading.

The system has three stages. First, a reference database of horizon profiles is synthesized from the Copernicus GLO-30 DEM using ray-casting. Second, a neural network segments the sky boundary from a query photograph. Third, a two-phase matching engine aligns the extracted profile against the database to determine coordinates.

3.2 Horizon Database Generation

We use the HORAYZON ray-casting library [?] to compute the horizon profile at each viewpoint in our study region. Each profile is a 1D vector of elevation angles:

$$V = \{h_1, h_2, \dots, h_{1440}\} \quad (3-1)$$

sampled at 0.25° azimuth intervals across a full 360° sweep. HORAYZON includes an optional correction for atmospheric refraction, which we enable during generation.

Fog and haze are common in the Khumbu valleys and can obscure distant peaks entirely. A single horizon computed at one fixed search radius would fail under these conditions. We therefore compute three horizons per viewpoint at different maximum search radii—global, local, and restricted—so that the matching engine can select the profile that best reflects the observer’s actual visibility. The three tiers are stored for each viewpoint and evaluated during matching.

3.3 Skyline Extraction

We use a U-Net with a MobileNet-V3 encoder to segment query photographs into sky and terrain [?, ?]. The model is pre-trained on GeoPose3K [?], which provides over 3,000 labeled alpine mountain images, and then fine-tuned on synthetic renderings of the Khumbu region generated from the DEM. These synthetic images are augmented with procedural fog textures to better represent local weather. A Canny edge detector [?] serves as a fallback when the neural network produces low-confidence output.

Figure 3-1 Raycasting to find the expected horizons at a given viewpoint using progressive search radii.

Figure 3-2 Pipeline for skyline extraction from photographs.

3.4 Matching Engine

Because the camera heading is unknown, the extracted skyline must be compared against every angular offset of every reference profile in the database. A brute-force search is prohibitively slow, so we split the matching into two phases.

3.4.1 Phase 1: Sliding-Window Euclidean Search

The query profile is treated as a subsequence that slides across each 1440-point reference vector. We compute the Euclidean distance at every offset using FFT-based cross-correlation, which reduces the per-profile cost from $O(Nm)$ to $O(N \log N)$ [?, ?]. We deliberately skip z-normalization so that the absolute elevation values—which carry strong discriminative signal in mountain terrain [?—are preserved. This stage produces a ranked short-list of candidate viewpoints.

3.4.2 Phase 2: DTW Refinement

Differences in camera focal length and tilt introduce non-linear distortions that Euclidean distance cannot absorb. Dynamic Time Warping handles these elastic deformations by allowing the alignment path to stretch or compress locally. We constrain the computation with Sakoe-Chiba bands and apply early abandoning and pruning to discard poor candidates before the full cost matrix is evaluated [?].

3.5 Validation

We evaluate the system in two ways. On the synthetic side, we hold out a subset of generated horizons and measure localization accuracy on this test split. For real-world evaluation, we use geotagged panoramas from Google Street View’s Everest trail coverage [?]. The Khumbu trails have sufficient Street View data for this purpose. Rather than matching directly against full panoramas, we decompose each one into standard perspective photographs and run the pipeline on these individual frames, comparing the predicted coordinates against the known GPS tags.

Figure 3-3 Detailed architecture of the matching engine.

4 EXPECTED RESULTS

The final system will take a mountain photograph as input and return a predicted location and heading. For each query, the output includes the top-ranked matching coordinates on a map, the extracted skyline overlaid against the best-matching database horizon, and a confidence score derived from the DTW alignment cost. Testing on Google Street View imagery from the Khumbu trails [?] will measure how often the correct grid cell appears in the top- k results.

Figure 4-1 Expected output interface showing the input photograph, predicted location on a map, profile alignment, and ranked candidate list.

5 FEASIBILITY ANALYSIS

5.1 Overview

This chapter examines whether the proposed system is practical to build and run. We work through the data requirements, storage costs, and computational loads for each stage of the pipeline, then describe the validation strategy.

5.2 Elevation Data and Database Generation

5.2.1 Choice of DEM

A Digital Elevation Model stores terrain height as a grid of cells. We need this grid to reconstruct 3D mountain horizons for matching.

We selected Copernicus GLO-30, which covers the globe at 30 m resolution and was released by the European Space Agency in 2020 [?, ?]. In rugged alpine terrain, GLO-30 has a measured root-mean-square vertical error of about 2.34 m [?]. Comparative studies in high-relief mountain ranges confirm that it provides a more consistent surface than older alternatives like AW3D30 [?].

We considered two other models but found them unsuitable:

- **SRTM and NASADEM:** Both have data gaps in steep areas caused by radar shadow and layover [?].
- **FABDEM:** Designed to remove forest canopy and buildings from the GLO-30 base. Since the high-altitude Khumbu terrain consists almost entirely of rock and ice with negligible vegetation or human structures, the canopy-removal features of FABDEM offer no functional advantage over the base model for this application.

5.2.2 Data Volume and Computational Requirements

The study area covers the main trekking corridors of Khumbu (40 km × 30 km). To prevent horizons from being clipped at the edges, we add a 50 km buffer around the target area—matching the maximum ray-casting distance to ensure all visible peaks are included—giving a total bounding box of roughly 140 km × 130 km. At 30 m spacing, this contains about 20 million elevation cells and requires 50–80 MB of storage.

The reference database is built by placing viewpoints on an 80 × 60 grid spaced 500 m apart (chosen as a balance between localization resolution and database size, sufficient for identifying major trail sections), giving 4,800 viewpoints. At each one, we cast 1,440 rays at 0.25° intervals:

$$360^\circ / 0.25^\circ = 1440 \text{ rays.}$$

Each ray steps through the DEM out to a maximum distance of 50 km, requiring roughly 1,667 elevation lookups per ray. So one profile costs:

$$1440 \times 1667 \approx 2.4 \times 10^6 \text{ elevation checks.}$$

Across all 4,800 viewpoints:

$$4800 \times 2.4 \times 10^6 \approx 11.5 \times 10^9 \text{ total checks.}$$

This is large but straightforward. Array-based processing on a modern computer finishes the entire generation in minutes [?].

5.2.3 Database Storage

Each profile is stored as 1,440 single-precision floats (4 bytes each):

$$1440 \times 4 = 5760 \text{ bytes} \approx 5.6 \text{ KB per profile.}$$

We store three versions of each profile, computed at different maximum search radii (global, local, and restricted) to handle varying visibility. All three tiers are stored per viewpoint and available for matching:

$$5.6 \text{ KB} \times 3 \approx 16.9 \text{ KB per viewpoint.}$$

For all 4,800 viewpoints:

$$4800 \times 16.9 \text{ KB} \approx 79 \text{ MB.}$$

This fits comfortably in system memory. A direct in-memory search is sufficient—no external database or indexing library is needed.

5.3 Two-Stage Matching Pipeline

The matching pipeline has two stages: a fast global search followed by fine alignment.

5.3.1 Step 1: FFT-based Sliding-Window Search

A query photograph produces a partial skyline of length m (for example, $m = 240$ points for a 60° field of view at 0.25° resolution). Since the camera heading is unknown, this partial profile must be compared against every angular offset of each 4,800 reference profiles ($N = 1440$ points each).

A naive sliding-window comparison costs $O(N \times m)$ per profile. We speed this up with FFT-based cross-correlation, which computes the full Euclidean distance profile in $O(N \log N)$ time [?]. We deliberately skip z-normalization to preserve the absolute elevation values in the signal.

The per-profile operation counts illustrate the speedup:

- **Brute force:**

$$N \times m = 1440 \times 240 = 345,600 \text{ operations.}$$

- **FFT-based:**

$$N \log_2 N \approx 1440 \times 10.49 \approx 15,106 \text{ operations.}$$

Across all 4,800 profiles, brute force requires about 1.66 billion operations; the FFT approach needs roughly 73 million—a 23-fold reduction. Based on the documented throughput of the HORAYZON engine [?] and the complexity of the MASS algorithm [?], this first stage is projected to complete in under one second on standard consumer hardware.

5.3.2 Step 2: DTW Refinement

The first stage produces 50–100 candidate viewpoints. This shortlist represents the top 1–2% of the 4,800 global viewpoints by L2 distance, providing a sufficient subset for elastic alignment while maintaining low computational overhead. Camera focal length and tilt can stretch or compress the captured skyline in ways that Euclidean distance cannot handle. We therefore re-rank the candidates using Dynamic Time Warping (DTW) [?], which is consistent with established terrain-matching pipelines [?].

Standard DTW has $O(m^2)$ complexity [?]. We apply two optimizations:

1. **Sakoe-Chiba band:** The warping path is confined to a diagonal window of width w . Setting $w = 24$ (10% of the query length, a standard heuristic for time-series matching [?]) reduces the cost per candidate to $O(m \times w)$.
2. **Early abandoning and pruning:** Paths exceeding the current best distance are terminated immediately [?].

Per candidate:

$$m \times w = 240 \times 24 = 5760 \text{ operations.}$$

Evaluating 100 candidates costs 576,000 operations—negligible on any modern processor.

5.4 Extraction and Validation

5.4.1 Sky Segmentation Model

We use a U-Net with a MobileNet-V3 backbone for sky segmentation [?, ?]. A Canny edge detector [?] serves as a fallback if the neural network fails. Other recent models such as YUNet [?] provide useful benchmarks for comparison.

The model is pre-trained on GeoPose3K [?], which contains over 3,000 alpine mountain images with known camera poses. Since those images come from the European Alps, we fine-tune on synthetic Khumbu terrain generated from the DEM and augmented with fog textures [?].

5.4.2 Ground-Truth Verification

For real-world testing, we use panoramas from the 2014 Google Street View Trekker campaign along the Lukla–Everest Base Camp trail [?]. This collection has over 45,000 panoramas with verified GPS coordinates.

We do not match against full panoramas directly. Instead, we cut each panorama into perspective photographs (e.g., 12 crops at 30° intervals, each covering a 60° horizontal field of view) and run the pipeline on these individual frames. The validation strategy includes:

- **Preventing data leakage:** Train/test splits are made at the panorama level so that overlapping crops from the same physical location do not appear in both sets.
- **Correcting distortion:** Perspective crops are re-projected from equirectangular format into planar views.
- **Weather categories:** Test images are grouped by visibility to evaluate each database weather tier.

5.5 Implementation and Resources

5.5.1 System Specifications

The full reference database is about 79 MB. We store horizon profiles in a structured in-memory index and search them directly via FFT, without relying on external vector-indexing libraries.

With a 500 m grid spacing, localization accuracy is measured by correct grid-cell assignment. The heading estimation is constrained by the $\pm 0.25^\circ$ angular resolution of the database profiles, which represents the theoretical resolution limit of the sampling interval.

5.5.2 Project Timeline

DEM processing, database generation, and matching are all computationally light and fit within a 60-day schedule. Most of the effort goes into downloading and preparing the Street View validation set, fine-tuning the segmentation model, and running the evaluation. To avoid API rate limits, we process a representative subset of several hundred panoramas, yielding several thousand test frames.

5.6 Conclusion

The computational, storage, and data requirements of the proposed system are manageable within the available resources and timeline.